Appello TAP del 01/09/2026

Avete a disposizione **2 ore (2 ore e 30 minuti per chi ha DSA)** per svolgere la prova. La prova è composta da:

- **Quiz** su AulaWeb (11 punti)
- Esercizio di **programmazione** (11 punti)
- Esercizi di **testing** (6 + 5 = 11 punti)

La somma totale è **33 punti**, ma è necessario arrivare almeno a **5 punti su ciascuna delle tre parti** per passare l'esame.

Suggerimento: leggete **tutto** il testo prima di buttarvi a implementare "cose": la specifica dei test dell'esercizio 2 può aiutarvi a capire fin da subito cosa implementare (e cosa **NON** fare).

Per la consegna, caricate su AulaWeb la **solution "zippata"**, ovvero zippate la cartella che contiene il file `.sln` e i progetti, *dopo aver eseguito il Clean*. Codice che non compila o che non passa il `SanityCheck` (presente anche in versione `.txt` su AulaWeb) non verrà considerato. Dopo aver caricato la solution, per sicurezza, scaricate il vostro zip, scompattatelo in un'altra cartella e verificate che compili tutto.

Esercizio 1 (11 punti)

In un progetto *class library* implementare l'extension method

`TakeAccepted<T>`

che, dati:

- una sequenza (*potenzialmente infinita*) di elementi di tipo `T`, `source`
- un predicato `predicate`, di tipo `Func<T, bool>`, che decide se un elemento deve essere restituito
- un intero non negativo `maxAccepted`

restituisce, nello stesso ordine in cui appaiono in `source`, al massimo i primi `maxAccepted` elementi per cui `predicate(...)` restituisce `true`.

Per esempio, se `source` fosse

```
[1, 2, 3, 4, 5, 6, 7]
```

`predicate(x)` restituisse `true` esattamente per i numeri pari, e `maxAccepted == 2`, allora

```
source.TakeAccepted(x => x % 2 == 0, 2)
```

restituirebbe la sequenza

```
[2, 4]
```

Il metodo deve fermarsi appena ha trovato `maxAccepted` elementi accettati. Nell'esempio precedente, quindi, non dovrebbe essere necessario controllare gli elementi `5`, `6` e `7`.

Se `maxAccepted == 0`, il metodo restituisce la sequenza vuota. In questo caso, anche enumerando il risultato, non deve essere necessario ispezionare elementi di `source` né chiamare `predicate`.

Se `maxAccepted < 0`, il metodo dovrà sollevare `ArgumentOutOfRangeException` **immediatamente**, senza che sia necessario enumerare la sequenza restituita.

Non è richiesto di sollevare eccezioni nel caso `source` o `predicate` siano `null`: nel testo si assume l'uso dei Nullable Reference Types e una chiamata corretta al metodo.

Vincoli

- Il metodo non deve enumerare l'intera sequenza se non necessario.
- La sequenza `source` deve essere enumerata **una sola volta** per ogni enumerazione della sequenza restituita.
- Nell'implementazione di `TakeAccepted` è vietato usare operatori LINQ standard o metodi di librerie esterne che risolvano direttamente l'esercizio al posto vostro, in particolare `Where`, `Take`, `TakeWhile` o metodi analoghi.
  - NB: questo divieto riguarda solo l'implementazione di `TakeAccepted`, nel codice dei test non c'è nessun vincolo

Sanity check

Lo scopo di questo test è solo verificare che la *signature* del vostro extension method sia abbastanza simile a quanto atteso da permettere di compilare ed eseguire i test.

```
[Fact]
public void SanityCheck() {
    IEnumerable<int> x = Array.Empty<int>().TakeAccepted(_ => true, 2);
    _ = new List<string> { "a", "b" }.TakeAccepted(s => s.Length == 1, 1);
    _ = new double[] { 3.14, 2.71 }.TakeAccepted(d => d > 0, 0);
}
```

Esercizio 2 (6 punti)

Implementare, usando xUnit, i seguenti test relativi all'esercizio 1.

I test devono verificare che:

1. Invocando il metodo con `maxAccepted == -1` venga lanciata immediatamente l'eccezione `ArgumentOutOfRangeException`.

2. Dato `source` l'array vuoto di interi, un predicato che accetta tutti gli elementi e `maxAccepted == 3`, il risultato sia la sequenza vuota.
3. Dato `source` l'array `[1, 2, 3, 4, 5, 6]`, un predicato che accetta i numeri pari e `maxAccepted == 2`, il risultato sia la sequenza `[2, 4]`.
4. Dato `source` la lista `["", "qui", "quo", "qua", "nonna papera"]`, un predicato che accetta le stringhe che iniziano con `q` e `maxAccepted == 10`, il risultato sia la sequenza `["qui", "quo", "qua"]`.
5. Dato `source` l'array `[1, 3, 5, 7]`, un predicato che accetta i numeri pari e `maxAccepted == 3`, il risultato sia la sequenza vuota.
6. Dato `source` l'array `[10, 20, 30, 40]`, un predicato che accetta tutti gli elementi e `maxAccepted == 0`, il risultato sia la sequenza vuota.

Esercizio 3 (5 punti)

Implementare i seguenti test aggiuntivi per l'extension method `TakeAccepted`.

1. Verificate che invocando il metodo con `source == [10, 20, 30, 40]`, `maxAccepted == 2` e un mock di `Func<int, bool>` configurato per accettare tutti gli elementi, ed enumerando completamente il risultato, il predicato venga chiamato esattamente due volte, una su il valore `10` e l'altra sul valore `20`

2. Verificate che invocando il metodo con `source == [11, 22, 33, 44, 55]`, `maxAccepted == 5` e un mock di `Func<int, bool>` configurato per accettare i numeri dispari, enumerando solo i primi due valori del risultato il predicato venga chiamato esattamente tre volte, sui valori `11`, `22` e `33`

3. Scrivere un test parametrico

   ```
   TakeAcceptedInfiniteTest(int howMany, int divisor, int remainder)
   ```

   che usi come `source` la sequenza infinita dei numeri interi non negativi `[0, 1, 2, 3, ...]` e un predicato `p(x)` definito come:

   ```
   x % divisor == remainder
   ```

   Il test deve:

   - invocare `TakeAccepted` con `maxAccepted == howMany`
   - enumerare completamente il risultato con `ToArray()`
   - verificare che il risultato contenga esattamente gli `howMany` primi numeri interi non negativi congrui a `remainder` modulo `divisor`

   Per esempio, con `howMany == 4`, `divisor == 3` e `remainder == 1`, il risultato atteso è:

   ```
   [1, 4, 7, 10]
   ```

   Verificate che il test parametrico passi almeno per le triple:

   - `4, 3, 1`
   - `5, 2, 0`

- `6, 5, 4`

Nota: ovviamente `remainder` deve essere sempre strettamente minore di `divisor`. Assumiamo che tutte le annotazioni `InlineData` rispettino questo vincolo. La nuova versione di *xUnit* permetterebbe di rendere un test "inconclusivo" in queste situazioni, ma la versione che abbiamo a disposizione adesso non ha un supporto per questo genere di vincoli.